

```

import pandas as pd
df = pd.read_csv("/content/sample_data/cleaned_dataset.csv")

#print(df.isnull().sum())
#print( df.isna().sum())
print((df == 0).sum())

```

obs_id	1
class	0
default	4043
x1	1
x2	1
x3	1
x4	1
x5	1
x6	1
x7	1
x8	1
x9	1
x10	1
x11	1
x12	1
x13	1
x14	1
x15	1
x16	1
x17	1
x18	1
x19	1
x20	1
x21	1
x22	1
x23	1
x24	1
x25	1
x26	4154
time	0
year	0
testing_set	2961
training_set	1250

```

dtype: int64

```

Question 1: Analyze class imbalance (default vs non-default).

```

#Default(imbalanced)
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import recall_score

```

```

X = df.drop('default', axis=1)
y = df['default']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify = y)

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)

recall = recall_score(y_test, y_pred)
print("Recall(Imbalanced):", recall)

#Balanced
rf = RandomForestClassifier(n_estimators=100, random_state=42,
class_weight='balanced')
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)

recall = recall_score(y_test, y_pred)
print("Recall(Balanced):", recall)

Recall(Imbalanced): 0.058823529411764705
Recall(Balanced): 0.08823529411764706

```

Question 2: Build a baseline Logistic Regression model.

```

df = pd.read_csv("/content/sample_data/cleaned_dataset.csv")
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score

X = df.drop('default', axis=1)
y = df['default']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify = y)

Lr = LogisticRegression()
Lr.fit(X_train, y_train)

y_pred = Lr.predict(X_test)

print("Classification Report:\n", classification_report(y_test,
y_pred))

```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
0	0.96	1.00	0.98	809
1	0.00	0.00	0.00	34
accuracy			0.96	843
macro avg	0.48	0.50	0.49	843
weighted avg	0.92	0.96	0.94	843

Confusion Matrix:

```
[[808  1]
 [ 34  0]]
```

Accuracy: 0.9584816132858838

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

Question 3: Try ensemble methods: Random Forest & XGBoost.

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score
from xgboost import XGBClassifier
```

```
X = df.drop('default', axis=1)
y = df['default']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify = y)
```

```
rf = RandomForestClassifier(n_estimators=100, random_state=42,
class_weight = 'balanced')
rf.fit(X_train, y_train)
```

```

y_pred = rf.predict(X_test)
y_prob_rf = rf.predict_proba(X_test)[:, 1]

print("Classification Report:\n", classification_report(y_test,
y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))

xgb = XGBClassifier(n_estimators=100, random_state=42)
xgb.fit(X_train, y_train)

y_pred = xgb.predict(X_test)

print("Classification Report:\n", classification_report(y_test,
y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))

```

Classification Report:

	precision	recall	f1-score	support
0	0.96	1.00	0.98	809
1	1.00	0.09	0.16	34
accuracy			0.96	843
macro avg	0.98	0.54	0.57	843
weighted avg	0.96	0.96	0.95	843

Confusion Matrix:

```
[[809  0]
 [ 31  3]]
```

Accuracy: 0.963226571767497

Classification Report:

	precision	recall	f1-score	support
0	0.97	1.00	0.98	809
1	0.62	0.15	0.24	34
accuracy			0.96	843
macro avg	0.80	0.57	0.61	843
weighted avg	0.95	0.96	0.95	843

Confusion Matrix:

```
[[806  3]
 [ 29  5]]
```

Accuracy: 0.9620403321470937

Question 4: Evaluate using AUC-ROC and classification report.

```
from sklearn.metrics import roc_auc_score

#AUC-ROC

print("\nROC-AUC Score:", roc_auc_score(y_test, y_prob_rf))
```

ROC-AUC Score: 0.8030793281465862

Question 5: Perform SMOTE or similar if class imbalance exists

```
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score, recall_score

X = df.drop('default', axis=1)
y = df['default']

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

X_train, X_test, y_train, y_test = train_test_split(X_resampled,
y_resampled, test_size=0.2, random_state=42)

Lr = LogisticRegression()
Lr.fit(X_train, y_train)

y_pred = Lr.predict(X_test)

print("Classification Report:\n", classification_report(y_test,
y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
0	0.77	0.83	0.80	813
1	0.81	0.75	0.78	805
accuracy			0.79	1618
macro avg	0.79	0.79	0.79	1618
weighted avg	0.79	0.79	0.79	1618

Confusion Matrix:

```
[[674 139]
 [203 602]]
```

```
Accuracy: 0.788627935723115
Recall: 0.7478260869565218
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (`max_iter`) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
`n_iter_i = _check_optimize_result(`

Question 6: Feature scaling and encoding.

Answer: **Feature Scaling** Here, the model will try to predict whether the loan defaults or not.

In this dataset, the features are $x_1, x_2, x_3, \dots, x_{26}$ and we use these as inputs and the target variable is default.

Encoding

For encoding iam using Label Encoder: `from sklearn.preprocessing import LabelEncoder`

Target variable : `y = df['default']`

Label Encoding a categorical column : `label_encoder = LabelEncoder() df['class'] = label_encoder.fit_transform(df['class'])`

Selecting feature columns (example) : `X = df[['x1', 'x2', 'x3', 'x4', 'x5']]`